(Near) Bit-Optimal Representations of ADT in Haskell

Giuliano Gorgone, Nan Wang
(alphabetical order)

Saturday 14th March, 2026

Abstract

In this initial version of project, we aim to implement *succinct trees*—data structures that provide almost information-theoretically optimal space bounds for tree representations. Furthermore, we aim leverage the Haskell type system to automatically derive (near) bit-optimal representations for arbitrary algebraic data types. Our ultimate goal is to provide a transparent interface that seamlessly utilizes the most efficient bit-level layout and its corresponding operations, eliminating the need for complex manual bit-manipulation.

In preliminary experiments, our succinct tree implementation, albeit in a beta stage, achieves significantly faster traversal times—often by one or more orders of magnitude—while using at least 2× less memory than a conventional pointer-based baseline.

Contents

1 Introduction **2**

2 Succinct Trees via DFUDS **2**

3 Implementation **3**

3.1 Basic Tree Baseline 4

3.2 DFUDS Succinct Tree 4

3.3 Preliminary Benchmarks 6

4 Simple Tests **8**

5 Conclusion **8**

5.1 Current Limitations 8

5.2 Future Work 9

Bibliography **9**

1 Introduction

The core idea of this project is to use the Haskell type system to automatically derive succinct, almost information-theoretically optimal runtime representations for standard algebraic data types (products and sums) and common built-in types such as `Int` and `Bool`. In functional programming, complex domains are often modeled through highly expressive datatypes, but standard implementations represent these values as large graphs of heap-allocated nodes linked by pointers. Our goal is to eliminate the need for manual bit-manipulation and instead automatically synthesize compact, bit-level layouts directly from datatype definitions.

To achieve this, we rely primarily on `GHC.Generics` to inspect the structure of user-defined types, aiming for a robust initial implementation that avoids Template Haskell entirely. By defining a `Succinct` type class—backed by Haskell's type and data families—the library can derive specialized combinators (such as customized `fmap` and `fold` operations) that allow developers to query, navigate, and transform data while it remains largely in a compressed, contiguous form. This approach is particularly relevant for computational logic applications such as provers or model checkers, which routinely manipulate huge trees, state sets, and transition relations and therefore suffer from high memory usage and frequent garbage-collection pauses. A concrete objective of this work is to evaluate whether our succinct backend can significantly reduce the memory footprint and improve traversal performance of existing generic zipper-style libraries such as GenZ.

2 Succinct Trees via DFUDS

Rather than relying on scattered[1], heap-allocated nodes, a DFUDS tree encodes structure and data in a sequential layout. The key advantage over "vanilla" pointer-based trees is that DFUDS cleanly disentangles the tree's topological shape from the payload values it stores.

This separation allows us to use an information-theoretically optimal representation for the topology alone. The number of structurally distinct binary trees with n nodes is given by the n-th Catalan number C_n, and the theoretical minimum number of bits required to distinguish between these shapes is $\log_2 C_n$, which asymptotically approaches $2n + o(n)$. DFUDS meets this optimal bound for the structural component of the tree.

We base our succinct tree representation on the Depth-First Unary Degree Sequence (DFUDS) encoding introduced by Benoit et al. [BDM+05]. DFUDS encodes the topology of a rooted tree in $2n + o(n)$ bits while supporting key navigational queries—such as parent, i-th child, subtree size, and lowest common ancestor—in $O(1)$ time. This makes DFUDS an ideal foundation for succinct tree structures and for zipper-like navigation over large models. In our Haskell prototype, we store the DFUDS bitstring inside a `Poppy512` index from the `hw-rankselect` library, while the node payloads are stored contiguously in an unboxed vector. This clean separation between topology and payload is what allows us to approach the information-theoretic minimum for space and still obtain highly cache-friendly traversals.

Remark 1. *The following explanation is intentionally informal and focuses on intuition rather than full formalism.*

[1] With this, we mean that the data is stored non-contiguously in memory. See Cache Locality.

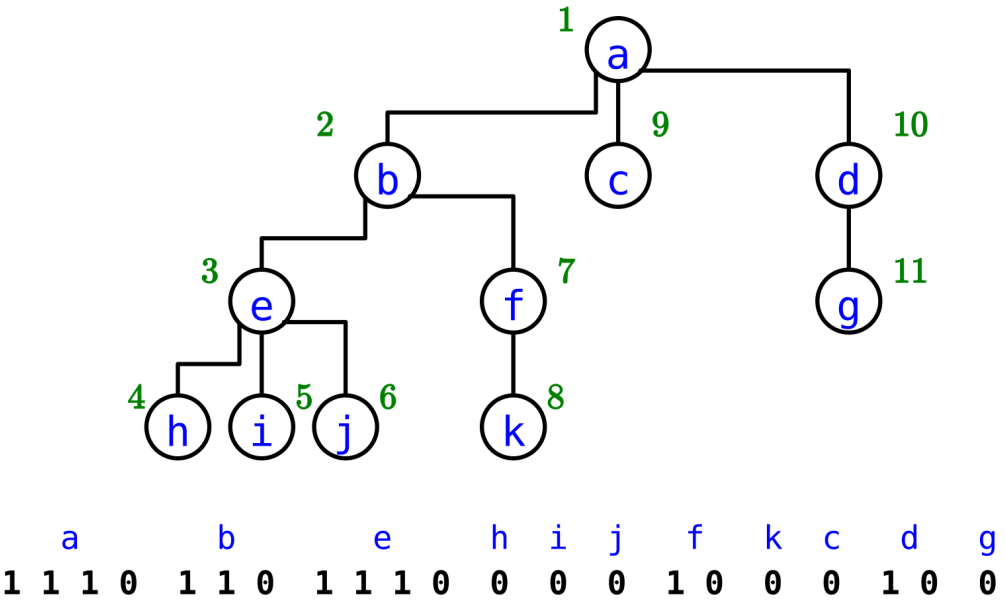

Figure 1: Example of DFUDS (Depth-First Unary Degree Sequence) encoding of a rooted tree, adapted from Manzini [Man25].

The encoding is as conceptually simple as efficient in practice. We traverse the tree in depth-first pre-order: starting from the root, we visit its first child and all of its descendants, then the second child, and so on. For every node with d children, we append d `1`s followed by a single `0` to a global bitstring. For example, in Figure 1, the root node **a** has three children and contributes `1110`; node **b** has two children and contributes `110`; node **e** has three children and contributes `1110`; each leaf (such as **h**, **i**, or **j**) contributes a single `0`; and so on for the remaining nodes. Concatenating these local codes in the order of the traversal yields a single contiguous DFUDS bitstring that fully captures the parent–child structure of the original tree without storing any explicit pointers.

In practice, implementations often prepend one artificial `1` at the beginning of the DFUDS string. This ensures enables constant-time navigation via fast `rank` and `select` operations on the underlying bitvector. Future work *could* explore complementing DFUDS (which is naturally depth-first) with LOUDS (Level-Order Unary Degree Sequence). Intuitively, LOUDS plays the same role as DFUDS but for breadth-first order: it provides a succinct encoding of the same tree topology, while arranging nodes in level order rather than depth-first order, enabling efficient BFS traversals within the same succinct framework.

3 Implementation

In this section, we present a minimal implementation of our DFUDS representation for trees and evaluate its performance against a standard pointer-based baseline.

3.1 Basic Tree Baseline

We first define a standard, pointer-based binary tree to serve as our baseline. It is similar to a standard algebraic datatype definition, but we enforce strictness using bang patterns (`!`). By manually discarding Haskell's default lazy evaluation on the structural fields, we aim to prevent excessive thunk buildup during evaluation.

```
data Tree a = Leaf | Node !a !(Tree a) !(Tree a)

foldlTree :: (b -> a -> b) -> b -> Tree a -> b
foldlTree f z Leaf           = z
foldlTree f z (Node v l r)   =
  let z1 = f z  v
      z2 = foldlTree f z1 l
  in  foldlTree f z2 r
```

However, simply adding bang patterns might not be enough for accurate benchmarking. In Haskell, generating a large structure dynamically (e.g., millions of nodes) results in lazy thunks being created rather than fully allocated objects. If we were to benchmark the traversal directly, the measured time would incorrectly include the allocation and evaluation time of the tree itself.

To prevent this, we instantiate the `NFData` (Normal Form Data) typeclass from the `deepseq` library. This allows our benchmarking framework to deeply force the entire tree into memory (Normal Form)[2] *before* the execution timer starts, ensuring we strictly measure the traversal algorithm and not the garbage collector or the tree initialization overhead.

```
instance NFData a => NFData (Tree a) where
  rnf Leaf         = ()
  rnf (Node v l r) = rnf v `seq` rnf l `seq` rnf r

countNodes :: Tree a -> Int
countNodes Leaf = 1
countNodes (Node _ l r) = 1 + countNodes l + countNodes r
```

We also provide a generator function `mkTree`, which creates a perfectly balanced tree of depth d which is *strictly evalueated*:

```
mkTree :: Int -> Int -> Tree Int
mkTree _ 0     = Leaf
mkTree !val d  = Node val (mkTree (val+1) (d-1)) (mkTree (val+2) (d-1))
```

3.2 DFUDS Succinct Tree

In our Haskell implementation, the disentanglement between topology and data is represented by two separate fields. The topology is encoded in `succStruct`, while the actual values are stored contiguously in `values`. For the bit-vector, we utilize `Poppy512` from the `hw-rankselect` library. `Poppy512` is a highly optimized, cache-friendly bit-vector, enabling $O(1)$ time complexity for `rank` and `select` queries.

```
data DFUDSTree a = DFUDSTree
  { succStruct :: !Poppy512
  , values     :: !(U.Vector a)
  , nNodes     :: !Int
  }

instance NFData (DFUDSTree a) where
  rnf (DFUDSTree bits vals _) = rnf bits `seq` rnf vals
```

[2] `https://hackage.haskell.org/package/deepseq`

As a running example, consider the following small tree:

```
example :: Tree Int
example =
  Node 609
    (Node 608 (Node 605 Leaf Leaf) Leaf)
    (Node 607 Leaf Leaf)
```

Its topology, in DFUDS form (with a leading artificial `1`), is encoded by the following bitstring:

`1 110 100 00`

and the corresponding payload vector contains the node values in DFS pre-order:

`[609, 608, 605, 607].`

To illustrate how DFUDS is constructed, we first present a naive list-based encoder. This implementation is intentionally inefficient and only serves to convey the core idea on small examples.

```
-- Naive DFUDS encoder for the topology, using a list of 0/1 bits.
getBits :: Tree a -> [Int]
getBits Leaf              = [0]
getBits (Node _ l r) =
  let d = degree l r
  in  replicate d 1 ++ [0] ++ getBits l ++ getBits r
  where
    degree Leaf Leaf                  = 0
    degree Leaf (Node _ _ _)          = 1
    degree (Node _ _ _) Leaf          = 1
    degree (Node _ _ _) (Node _ _ _) = 2
```

We then pack this list of bits into a `Poppy512` bitvector in a similarly naive way, by repeatedly updating an immutable `Vector Word64`. This is asymptotically suboptimal, but perfectly adequate for a beta.

```
bitsToPoppy :: [Int] -> (Poppy512, Int)
bitsToPoppy bits =
  let nBits  = length bits
      nWords = (nBits + 63) `div` 64
      zeroVec :: VS.Vector Word64
      zeroVec = VS.replicate nWords 0

      setOne acc (i, b) =
        case b of
          1 ->
            let wIx  = i `div` 64
                bIx  = i .&. 63
                oldW = acc VS.! wIx
                newW = setBit oldW bIx
            in  acc VS.// [(wIx, newW)]
          _ -> acc

      packed = foldl setOne zeroVec (zip [0..] bits)
  in  (makePoppy512 packed, nBits)
```

The stored value are extracted in the same DFS pre-order as used by `getBits`. We define a tree fold and use it to obtain an $O(n)$ conversion to a list.

```
toListDFS :: Tree a -> [a]
toListDFS t = reverse (foldlTree (\acc v -> v : acc) [] t)
```

We can now define a simple conversion function from a pointer-based tree to our `DFUDSTree`. It materializes the DFUDS topology as a list of bits, packs it into a `Poppy512` bitvector, and stores the payloads in an unboxed vector.

```
toDFUDS :: U.Unbox a => Tree a -> DFUDSTree a
toDFUDS t =
  let bits            = getBits t
      vals            = toListDFS t
      (pop, bitCount) = bitsToPoppy bits
  in  DFUDSTree
        { succStruct = pop
        , values     = U.fromList vals
        , nNodes     = bitCount
        }
```

This `toDFUDS` function should be viewed as an executable specification rather than an efficient implementation. Our final version will construct the topology bitvector directly, without going through intermediate lists or repeated vector updates.

Because the data payload is fully decoupled from the tree structure and stored in a contiguous unboxed vector, operations that only inspect values (such as folding) become extremely efficient. Instead of recursively chasing pointers through the heap, a DFUDS fold reduces to a cache-friendly linear scan:

```
foldlDFUDS :: U.Unbox a => (b -> a -> b) -> b -> DFUDSTree a -> b
foldlDFUDS f z tree = U.foldl' f z (values tree)
```

3.3 Preliminary Benchmarks

To compare traversal performance, we construct a binary tree of a user-specified depth using `mkTree`, and then convert this pointer-based tree into our succinct `DFUDSTree` representation. Reading the depth from standard input ensures that GHC cannot specialize or precompute the entire benchmark at compile time, so all work is performed at runtime.

We rely on `tasty-bench` to obtain stable runtime measurements. The benchmark uses the `env` function to separate data generation from the actual measurements: the tree and its DFUDS encoding are fully evaluated once and then reused across all benchmark runs for `evalBoxed`, `foldlTree`, and `foldlDFUDS`. This prevents the cost of constructing the input data from being folded into the reported timings and avoids aggressive compile-time constant folding optimizations by GHC. In addition, we use the `ghc-datasize` library to measure the precise heap footprint of both representations.

```
main :: IO ()
main = do
  putStrLn "Depth?"
  depthStr <- getLine
  let depth =
        case reads depthStr of
          [(d, "")] -> d
          _         -> error "Please enter a valid integer depth."

  let setupData = do
        vTree <- evaluate . force $ mkTree 0 depth
        let dTree = toDFUDS vTree
        -- force DFUDS as well, so both structures are fully allocated
        vTree `deepseq` dTree `deepseq` return (vTree, dTree)

  (v, d) <- setupData

  if depth <= 14
    then do
      sizeV <- recursiveSize $!! v
      sizeD <- recursiveSize $!! d

      printf "Vanilla Tree Size: %.2f MB\n"
```

```haskell
        (fromIntegral sizeV / (1024 * 1024) :: Double)
      printf "DFUDS Tree Size:   %.2f MB\n"
        (fromIntegral sizeD / (1024 * 1024) :: Double)
    else
      printf "Exact memory usage tracking disabled for large depths (ghc-datasize is too
          slow).\n"

  -- Force full evaluation BEFORE printing sums
  v `deepseq` d `deepseq` return ()

  printf "Sum result Vanilla:   %d\n" (foldlTree (+) 0 v)
  printf "Sum via DFUDS Foldl (should match Vanilla): %d\n" (foldlDFUDS (+) 0 d)

  defaultMain
    [ env (return (v, d)) $ \ ~(vanilla, dfuds) ->
        bgroup "Visit"
          [ bench "Vanilla Eval"
              $ nf (foldlTree (+) 0) vanilla
          , bench "DFUDS Foldl Array"
              $ nf (foldlDFUDS (+) 0) dfuds
          ]
    ]
```

Because Haskell is lazily evaluated, constructing a large tree such as `mkTree 0 depth` does not immediately allocate all nodes in memory. If we were to benchmark `foldlTree` or `foldlDFUDS` directly on these thunks, the reported time would conflate two costs: building the tree (and its DFUDS encoding) and actually traversing it. This would unfairly penalize the succinct representation, since its construction is more expensive than that of the pointer-based baseline.

To obtain meaningful traversal timings, we explicitly force both the vanilla tree and its DFUDS encoding into normal form before running the benchmarks, using `evaluate . force` and `deepseq`. In this way, the measured time isolates the cost of the traversal itself, excluding the one-off construction overhead of the underlying data structures.

For a balanced binary tree of depth 15, we obtain the following empirical results. The succinct DFUDS representation uses roughly half the memory of the vanilla pointer-based tree, while achieving an order-of-magnitude speedup in traversal time.

```
Depth?
15
Vanilla Tree Size: 1.00 MB
DFUDS Tree Size:   0.52 MB
Sum result Vanilla:   638979
Sum via DFUDS Foldl: 638979

All
  Visit
    Vanilla Eval:      OK (0.93s)
      3.47 ms ± 265 µs
    DFUDS Foldl Array: OK (0.21s)
      47.6 µs ± 3.9 µs

All 2 tests passed (1.14s)
```

More impressively, with a depth of 20, we get the following results, confirming that our implemetation, albeit suboptimal, is order of magnitudes faster in traversal time.

```
Depth?
20
Exact memory usage tracking disabled for large depths (ghc-datasize is too slow).
Sum result Vanilla:   28311555
Sum via DFUDS Foldl (should match Vanilla): 28311555
All
  Visit
    Vanilla Eval:      OK (1.05s)
      485  ms ±  25 ms
    DFUDS Foldl Array: OK (0.45s)
      1.26 ms ±  71 µs
All 2 tests passed (1.50s)
```

4 Simple Tests

We will use the `QuickCheck` library to automatically generate random test cases for our core operations.

Concretely, we will use QuickCheck for two purposes:

- **Fold order**: to verify that the DFUDS-based folds visit node payloads in the intended traversal order and produce the same result as the corresponding folds over the pointer-based tree.
- **Zipper navigation**: to check that our (succinct) zipper operations move correctly within the tree—e.g. navigating up, down, left, and right always lands on the expected node and preserves structural invariants.

5 Conclusion

This report describes (read, will describe) a library for automatically deriving succinct data structures using the Haskell type system. By separating structural topology from data payloads and encoding the topology using DFUDS (Depth-First Unary Degree Sequence), we obtain a compact representation of trees that avoids traditional pointer-heavy layouts. This separation allows traversals to operate over contiguous memory rather than following chains of heap pointers. As a result, many operations reduce to cache-friendly operation over bitvectors, dramatically improving memory locality and reducing garbage collection overhead.

5.1 Current Limitations

The current prototype has several limitations that will be addressed in the next development phase:

- **Only in the beta**, the DFUDS construction is implemented in a sub-optimal way, which is asymptotically inefficient. Our final version will construct the topology bitvector directly, without going through intermediate lists or repeated vector updates.

- The implementation currently supports only explicitly defined binary trees rather than arbitrary algebraic tree structures derived automatically via metaprogramming or any alternative.
- Navigational operations over the succinct representation remain limited. In particular, we aim to implement a Succinct Tree Zipper.

5.2 Future Work

Several directions remain for extending this work:

- **Generic Derivation**: Extend the system to automatically support arbitrary algebraic data types by deriving succinct representations from sums and products.
- **Computational Logic Applications**: Evaluate the library in practical settings such as model checking and measuring whether succinct representations improve memory usage and processing bounds.
- **LOUDS Representation (if we have enough time)**: Introduce support for the Level Order Unary Degree Sequence (LOUDS) representation to enable efficient breadth-first navigation and accelerate BFS-style algorithms.

References

[BDM+05] David Benoit, Erik D Demaine, J Ian Munro, Rajeev Raman, Venkatesh Raman, and S Srinivasa Rao. Representing trees of higher degree. *Algorithmica*, 43(4):275–292, 2005.

[Man25] Giovanni Manzini. Succinct tree representations. Slides from the FA25 School (ELICSIR, SchmidtSciences), February 2025.